

Hospital PDF System — End to End Flow

Complete lifecycle from scan to insurance portal submission, including network resilience.

PART A — UPLOAD FLOW (Scan to Storage)

Step 1: User Scans a Document

User opens the app, navigates to patient folder (e.g., Claim Forms), taps Scan. ML Kit opens in SCANNER_MODE_FULL. User captures one or more pages. ML Kit auto-detects edges, corrects perspective, removes shadows, whitens background.

Step 2: Android Post-Processes Each Page

App takes ML Kit output images and warps each one to true A4 canvas (1654 x 2339 at 200 DPI). Folder name decides color mode: ID/Insurance folders stay color, Reports/Prescriptions/Claims become grayscale. Each page encoded as JPEG quality 80. Pages wrapped into one PDF using PdfUtils.kt. Result: PDF at ~150-250 KB per page.

Step 3: Save Locally First (Always)

PDF written to app's local storage. Entry created in local Room database with status PENDING_UPLOAD. UI immediately shows the document as available under its folder with a small syncing indicator. User can close scanner, re-view, or scan again right away. No waiting.

Step 4: Background Upload via WorkManager

WorkManager schedules an upload job with NetworkType.CONNECTED constraint. When connection is available: uploads PDF to Cloudinary under authenticated delivery type, under path like patient_{id}/folder_{name}/doc_{timestamp}.pdf. On success: marks entry SYNCED in local DB, keeps local copy for short-term cache. On failure: retries with exponential backoff (30s, 1min, 5min, 15min). Badge in app header shows pending count.

Step 5: Cloudinary Stores Original

Cloudinary stores the PDF as-is. This is the source of truth. Never compressed or modified. Delivery is authenticated only; no public URLs for medical data. This satisfies DPDP requirement that PHI not be exposed via public endpoints.

PART B — DOWNLOAD FLOW (Merge, Compress, Deliver)

Step 1: User Taps Download

Two entry points: Download Folder button (single folder) or Download Patient button (all folders merged). Android shows immediate feedback: creates a download entry in local DB with status PREPARING, displays a non-blocking progress indicator, user can navigate away.

Step 2: Android Calls Cloud Run Service

Request sent to Cloud Run Mumbai endpoint (/folder-download or /patient-download). Payload includes: user auth token, patient ID, folder ID (or list), target size (5 MB or 10 MB). Android now polls status endpoint every 3-5 seconds (for patient downloads) or waits on the sync response (for folder downloads).

Step 3: Cloud Run Verifies and Checks Cache

Service verifies user has permission to access this patient. Computes content hash = SHA-256 of (all source PDF IDs + last-modified timestamps + target size). Queries Cloudinary for merged/{hash}.pdf. If found: returns that URL immediately, skips all processing. This is the common case for repeat downloads. If not found: proceeds to fresh processing.

Step 4: Fetch Source PDFs in Parallel

Service generates signed Cloudinary URLs for each source PDF. Downloads all of them in parallel (not sequentially) into /tmp directory of the container. Cloud Run provides fast in-region access to Cloudinary. Typically 10-30 seconds for a large patient on first run.

Step 5: Classify Each PDF

For each source PDF, service checks whether it has a real text layer (digital PDF) or is image-only (scan). Digital PDFs get light optimization only, keeping vector text sharp. Scanned PDFs are eligible for aggressive image compression. Classification stored as metadata hint for tier selection.

Step 6: Apply Defenses Before Compression

Perceptual hashing removes duplicate pages (same ID card across multiple folders, same discharge summary). Blank page detection removes empty backsides and separator sheets. For patient merges, budget is allocated proportionally: folder budget = 10 MB x (folder pages / total pages). These steps typically remove 15-30% of raw content before any compression runs.

Step 7: Merge Into One PDF

pikepdf combines all cleaned source PDFs into a single document. Order follows folder order as defined in the patient schema (ID, Claim, Reports, etc.). Bookmarks added per folder boundary for navigation in the final PDF.

Step 8: Run Tier Ladder to Hit Target Size

Service tries compression tiers top-down until output fits target:

Tier 0: Ghostscript /printer, 300 DPI color — highest quality

Tier 1: Ghostscript /ebook, 200 DPI color — default for small folders

Tier 2: Ghostscript /ebook, 150 DPI color — most folders land here

Tier 3: Ghostscript /ebook, 150 DPI grayscale — text-heavy folders

Tier 4: Ghostscript /screen, 150 DPI grayscale — large patients

Tier 5: MRC compression via archive-pdf-tools, 150 DPI — very large patients

Tier 6: MRC at 120 DPI — last resort, quality floor

Stops at first tier that fits. Never goes below Tier 6. If even Tier 6 fails, returns size-too-large error with suggestion to deselect folders.

Step 9: Quality Guardrails

Output PDF validated: page count matches expected, no corruption. Render-back check: low-DPI render of output compared against source for catastrophic differences. Audit log entry written: user ID, patient ID, timestamp, tier used, input size, output size, duration. Retained 7 years for DPDP and insurance audit.

Step 10: Upload Result to Cloudinary

Merged compressed PDF uploaded to Cloudinary under merged/{hash}.pdf with 30-day auto-delete tag. Uses authenticated delivery type. Signed URL generated with reasonable expiry (e.g., 1 hour). Cloudinary CDN now caches this file for any subsequent download of the same content.

Step 11: Return URL to Android

Cloud Run returns signed URL + content hash + final size to Android. Android updates its local download entry status from PREPARING to READY_TO_DOWNLOAD.

Step 12: Android Downloads from CDN

Android uses OkHttp with range-request support. Downloads the merged PDF from Cloudinary CDN (fast, regional). If connection drops mid-download, resumes from last byte via HTTP range request rather than restarting. Progress bar shows real bytes downloaded vs total. Runs inside WorkManager so user can close app.

Step 13: Local Cache + Open

Downloaded PDF stored in app cache folder, indexed by content hash. Notification fires: File ready to view. User taps notification, PDF opens in system viewer or in-app preview. Any future download of the same content hash opens instantly from local cache without hitting the network.

Step 14: User Submits to Insurance Portal

User shares the PDF via Android share sheet to insurance portal app, email, or WhatsApp. File size fits portal's upload limit (5 or 10 MB). Quality preserved because entire pipeline avoided double-compression — single compression from preserved original.

PART C — NETWORK RESILIENCE (Poor Connection Handling)

Upload Side — Invisible to User

Scan always saves locally first. User never waits for upload to finish. WorkManager retries with backoff. Badge shows pending count. If user scans 10 documents in offline mode, all 10 queue locally; when phone reconnects, they sync in the background. User's perception: instant save, regardless of network.

Download Side — Tolerable With Progress

Three visible stages shown to user: Preparing (Cloud Run processing, 5-60s), Downloading X MB of Y MB (CDN transfer), Ready. Download runs in WorkManager so user can navigate away. Resumable via HTTP range requests. Notification on completion. Cached downloads open instantly on repeat views.

Realistic Timing Expectations

Good internet: folder download 5-10s total; patient download first time 20-40s; patient cached 2-5s.

Poor internet: folder download 20-60s in background; patient first time 2-5 min in background with resumable progress; patient cached 30-90s (download only).

Offline: scanning works, uploads queue, previously-viewed downloads work from cache, new downloads show will retry when connected.

The Six UX Rules

1. Never block UI for network operations.
2. Show pending work via badges or indicators.
3. Notify on completion instead of spinner-watching.
4. Resume, don't restart, on network failure.
5. Cache what was seen for offline reuse.
6. Honest status messages over silent failures.

PART D — SUMMARY OF WHO DOES WHAT

Android App

Scanning, ML Kit FULL mode, A4 canvas warp, folder-aware color mode, JPEG 80 encoding, local cache, WorkManager upload queue, WorkManager resumable downloads, progress UI, notifications, local cache lookup by content hash.

Cloudinary

Storage of originals (untouched), authenticated delivery, signed URL generation, CDN for fast in-region delivery, storage of merged outputs with 30-day auto-delete, q_auto on single-PDF delivery URLs.

Cloud Run (Mumbai)

User auth verification, cache check via content hash, parallel fetch from Cloudinary, page classification, deduplication, blank removal, budget allocation, merge, tier ladder compression, quality validation, audit logging, write-back to Cloudinary, signed URL return.

External / Compliance

DPDP audit log in Postgres Mumbai (7-year retention), access log per download, all data in India region, no third-party PDF APIs used, HTTPS everywhere, authenticated-only delivery.